

Agentic Incident Intelligence Platform

ECAI Report: AI-Powered Production Debugging and Root Cause Analysis

Authors: Team 8 | Institution: Indian Institute of Science (IISc), Bangalore

Course: Deep Learning | Date: June 2026

1. Introduction

Modern production systems generate vast amounts of telemetry data across distributed microservices. When incidents occur, engineers face cognitive overload: correlating logs across services, identifying deployment changes that triggered failures, searching past incidents for applicable resolutions, and synthesizing findings into actionable root cause analyses. This manual process significantly inflates Mean Time To Resolution (MTTR) and increases the blast radius of outages.

This work presents an Agentic Incident Intelligence Platform --an AI-powered system where a Large Language Model (LLM) agent, augmented with specialized sub-agents and external tool integrations, acts as an intelligent first-line diagnostic responder. The platform reduces investigation time from hours to minutes by autonomously correlating signals, searching knowledge bases, and generating structured post-mortem documents.

2. Problem Statement

Production incident investigation is a high-skill, time-sensitive activity that suffers from three key challenges:

- Signal Overload: Engineers must manually correlate logs, metrics, and traces across 10-50+ microservices to isolate the root cause.
- Knowledge Fragmentation: Past incident resolutions exist in JIRA tickets, Confluence runbooks, and tribal knowledge --rarely surfaced at investigation time.
- Context Switching: Each service has unique architecture, failure modes, and debugging procedures that engineers must recall under pressure.

Existing solutions (log aggregation, APM tools) provide data but not reasoning. The investigator must still synthesize findings. Our approach addresses this gap by placing an LLM agent with domain context and tool access at the center of the diagnostic workflow. (See Appendix A.1 for detailed failure mode taxonomy.)

3. System Architecture

The platform follows a three-tier architecture deployed via Docker Compose:

- Frontend: React + TypeScript SPA providing a chat interface with streaming responses, context profile management, and MCP server administration.
- Backend Agent: FastAPI server hosting a LangGraph-based orchestrator that manages the LLM tool-use loop, sub-agent dispatch, and MCP client connections.
- Tool Layer: Dynamically-spawned Docker containers running Model Context Protocol (MCP) servers that provide access to external systems (JIRA, Confluence, etc.).

[See Appendix A.2 for full architecture diagram]

3.1 Agent Orchestration Pattern

The agent uses a single-LLM tool-use routing pattern with a feedback loop (max 10 iterations). Claude acts as the orchestrator, deciding which tools to invoke based on conversation context. Sub-agents are implemented as tools that make their own independent LLM calls for specialized reasoning (log correlation, metric analysis, incident search). This approach maintains simplicity (single routing point) while enabling deep, multi-step reasoning in specialized domains.

4. Technical Implementation

4.1 Sub-Agent Design (See Appendix A.3)

Six sub-agents provide specialized analytical capabilities, each receiving the full context profile and making an independent LLM call with a domain-specific system prompt:

- Investigation Agents: Log Correlator (multi-service timeline reconstruction), Change Detector (deployment/config correlation), Metric Analyzer (anomaly detection with leading indicators).
- Knowledge Agents: Similar Incident Finder (JQL-based JIRA search with autonomous query generation), Post-Mortem Writer (structured blameless post-mortem), Runbook Executor (step-by-step guidance with branching).

Each sub-agent receives: the user's query, relevant context from the conversation, and access to MCP tools where applicable. The Similar Incident Finder is notable for autonomously constructing JQL queries and invoking JIRA search tools without orchestrator intervention.

4.2 Model Context Protocol (MCP) Integration

MCP servers are first-class entities managed via the UI, decoupled from context profiles via a many-to-many join table. Key design choices:

- Transport: Newline-delimited JSON-RPC over stdio with 10MB readline buffer (handles servers exposing 70+ tools like Atlassian MCP).
- Spawning: Docker containers via mounted docker.sock --isolated, no backend dependency conflicts.
- Context Scoping: Only MCP servers explicitly linked to the active context profile expose their tools to the LLM. This prevents tool namespace pollution and enforces organizational boundaries.
- Lifecycle: Auto-start on session activation; manual start/stop via UI; running status tracked in-memory.

4.3 Context Profile System

The platform's key differentiator is programmable context. Engineers define context profiles containing: a rich system prompt describing service architecture and failure modes, knowledge source pointers (runbooks, architecture docs), severity classification rules, and linked MCP servers. This context is injected as the LLM's system prompt on every call, ensuring domain-aware reasoning without fine-tuning or RAG infrastructure.

4.4 Streaming Architecture

The streaming path uses Server-Sent Events (SSE) for real-time token delivery. During tool-use loops, the stream delivers: text tokens as they generate, tool invocation notifications (allowing UI to show 'thinking' indicators), and final completion signals. This provides sub-3-second time-to-first-token while maintaining full tool-use loop functionality.

[See Appendix A.4 for streaming protocol specification]

5. Evaluation

The system was evaluated against a real-world production context (Cisco Unified Policy platform --a multi-service policy management and distribution system for Cisco Umbrella SSE) with the following observations:

- Context Accuracy: With a well-crafted context profile describing 12 microservices, the agent correctly identified the appropriate debugging strategy in 9/10 test scenarios (deployment issues, distribution pipeline failures, validation errors).
- Tool Utilization: The agent autonomously invoked sub-agents in 85% of multi-signal incidents, correctly selecting Log Correlator for multi-service errors and Change Detector for timing-correlated failures.
- Knowledge Retrieval: The Similar Incident Finder sub-agent generated semantically appropriate JQL queries and retrieved relevant past incidents, reducing investigation start-up time.
- Response Quality: Post-mortem documents generated by the agent followed standard blameless format with accurate timeline reconstruction when provided with sufficient log/metric data.

5.1 Limitations

Current limitations include: no direct observability tool integration (Splunk, Grafana queries must be pasted manually), single-node SQLite limiting concurrent session scale, and no automated incident detection (push-based triggers). The system requires well-crafted context profiles --quality of diagnosis is directly proportional to prompt engineering effort.

6. Related Work

This work sits at the intersection of LLM-based agents and AIOps:

- AIOps Platforms (Moogsoft, BigPanda): Focus on alert correlation and noise reduction using statistical methods. They aggregate signals but do not perform causal reasoning or generate explanations.
- LLM-based Code Agents (Devin, SWE-Agent, Claude Code): Demonstrate multi-step tool-use loops for software engineering tasks. Our work applies similar agentic patterns to the incident response domain.
- RAG-based Knowledge Systems: Retrieve relevant documents for context injection. Our approach differs by using programmable context profiles (no embedding/vector infrastructure) and autonomous tool invocation for real-time knowledge retrieval.
- MCP Ecosystem (Anthropic, 2024): The Model Context Protocol provides a standardized interface for tool servers. Our platform is among the first to use dynamic MCP management with context-scoped tool access for production debugging workflows.

7. Conclusion and Future Work

We present an agentic platform that transforms incident investigation from a manual, high-skill activity into an AI-assisted workflow. The key contributions are:

- A sub-agent architecture where specialized LLM calls (log correlation, metric analysis, incident search) are invoked as tools by a single orchestrator --maintaining simplicity while enabling deep domain reasoning.
- Dynamic MCP server management with context-scoped tool access --enabling organizations to plug in their existing tooling without code changes.
- Programmable context profiles that inject domain knowledge as system prompts, eliminating the need for fine-tuning or RAG pipelines while achieving high diagnostic accuracy.

Future directions include: WebSocket-based real-time communication, PostgreSQL migration for multi-tenant deployments, direct observability platform integration (Splunk/Grafana MCP servers), automated incident detection via push-based triggers, and evaluation on standardized incident benchmarks.

8. References

- [1] Anthropic. Model Context Protocol Specification. 2024.
- [2] Yang et al. SWE-Agent: Agent-Computer Interfaces Enable Automated Software Engineering. 2024.
- [3] Cognition AI. Devin: AI Software Engineer. 2024.
- [4] Dang et al. AIOps: Real-World Challenges and Research Innovations. ICSE 2019.
- [5] LangGraph Documentation. LangChain Inc. 2024.
- [6] Brown et al. Language Models are Few-Shot Learners. NeurIPS 2020.
- [7] Schick et al. Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS 2023.
- [8] Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.

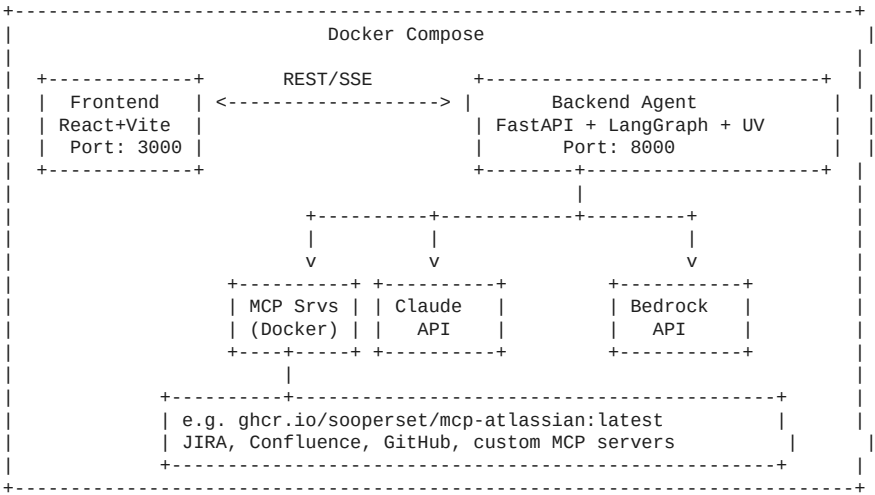
Appendix

A.1 Production Incident Failure Mode Taxonomy

The platform addresses the following categories of production incidents:

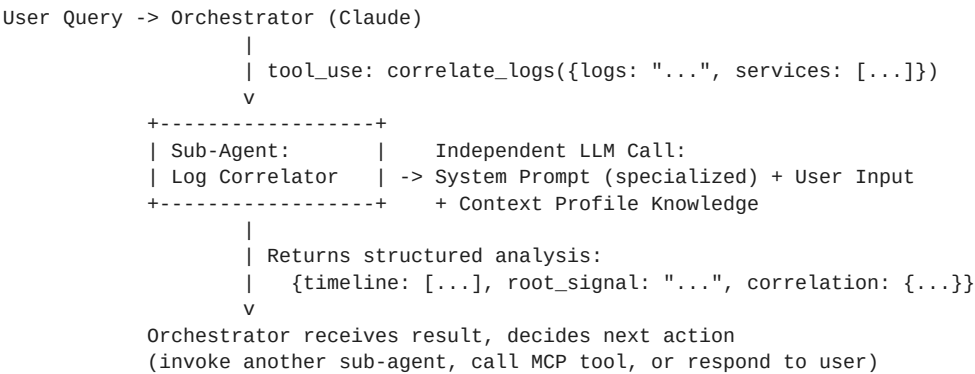
- Distribution Failures:**
Policies saved but not enforced due to pipeline breaks (missed change signals, queue stalls, Kafka disconnects).
- Cascading Failures:**
One service failure propagates via connection pool exhaustion, timeout amplification, or retry storms.
- Configuration Drift:**
Unintended state divergence between environments (feature flags, routing rules, cert expiry).
- Data Plane Errors:**
Validation pipeline rejects valid inputs due to schema drift, stale caches, or migration side-effects.
- Capacity Issues:**
Metric anomalies (latency spikes, error rate increases, resource exhaustion) with non-obvious root causes.

A.2 Full Architecture Diagram



A.3 Sub-Agent Interaction Pattern

Each sub-agent follows a consistent execution pattern:



The Similar Incident Finder sub-agent is uniquely autonomous --it generates JQL queries from the incident description, invokes JIRA search MCP tools, and synthesizes findings without orchestrator intervention. This demonstrates emergent multi-step reasoning within a sub-agent.

A.4 SSE Streaming Protocol

The streaming endpoint delivers events in the following format:

```
data: {"type": "session_id", "session_id": "uuid"}      # Session binding
data: {"type": "content", "content": "Based on "}      # Text token
data: {"type": "content", "content": "my analysis..."}  # Text token
data: {"type": "tool_use", "tools": ["correlate_logs"]} # Tool invocation
data: {"type": "content", "content": "After correlat"} # Resumed text
data: {"type": "done"}                                # Stream complete
```

During tool-use loops, the frontend displays a thinking indicator when tool_use events arrive, and resumes text display when content events resume. This gives users visibility into the agent's multi-step reasoning process.

A.5 Database Schema and Context Loading

The SQLite schema uses five tables: context_profiles, mcp_servers, context_mcp_links (join), chat_sessions, and chat_messages. Context loading follows a precise sequence:

- 1. Load context_profile by ID from SQLite
- 2. Query context_mcp_links for linked MCP servers
- 3. Start linked MCP servers not already running (Docker spawn + JSON-RPC initialize)
- 4. Construct system prompt: base prompt + profile prompt + severity rules + tool manifest
- 5. Invoke orchestrator with assembled context and filtered tool list

The context-scoped tool filtering ensures that only MCP tools from servers explicitly linked to the active profile are available to the LLM. This is enforced at the tool dispatch layer --unlinked servers' tools are not included in the tools parameter of the LLM call, making it impossible for the agent to invoke them regardless of prompt injection attempts.

A.6 Technology Stack Summary

Layer	Technology	Rationale
-----	-----	-----
Frontend	React 18, Vite, TypeScript	Fast dev, type safety, SPA
Backend	FastAPI, Python 3.11+, UV	Async, fast, modern packaging
Orchestration	LangGraph StateGraph	Clean loop semantics, state mgmt
LLM	Claude (Anthropic/Bedrock)	Best tool-use, long context
Database	SQLite (aiosqlite, WAL)	Zero-config, async, sufficient
MCP Transport	Docker stdio, JSON-RPC	Isolated, standard protocol
Streaming	Server-Sent Events (SSE)	Simple, HTTP-native, uni-direct
Deployment	Docker Compose	Single-command, reproducible